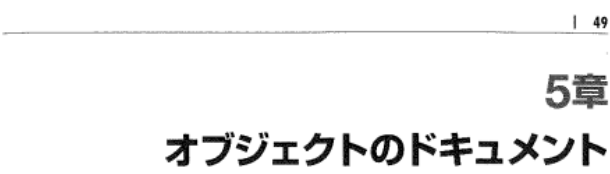


• 「Rパッケージ開発入門」

第5章roxygen (2nd editionでは16 Function documentation)

• • • 第5章roxygen / Function documentation • • •

この章では?やhelp()関数でアクセスされるFunction documentationについて学びます。



16 Function documentation

In this chapter, you’ll learn about function documentation, which users access with `?somefunction` or `help("somefunction")`. Base R provides a standard way of documenting a package where each function is documented in a **topic**, an `.Rd` file (“R documentation”) in the `man/` directory. `.Rd` files use a custom syntax, loosely based on LaTeX, and can be rendered to HTML, plain text, or pdf, as needed, for viewing in different contexts.

ドキュメントは、良いパッケージにおける最も重要な要素の1つです。ドキュメントなしでは、利用者はどのようにパッケージを使うのがわかりません。ドキュメントは、「未来の自分」にも（自分の書いた関数がどのような仕様であったのかを思い出すために）役立ち、そのパッケージを拡張する他の開発者のためにも役立ちます。

ドキュメントにはたくさんの形式があります。この章では、`?` や `help()` 関数でアクセスされるオブジェクトのドキュメントについて学びます。オブジェクトのドキュメントは、一種の参考文献です。辞書のような機能を持ちます。辞書は特定の単語の意味を知りたいときには役立ちますが、特定の状況に対して適切な単語を見つけるのには役立ちません。同様に、オブジェクトのドキュメントも、すでにオブジェクトの名前を知っていれば役立ちますが、特定の問題を解決するのに必要なオブジェクトを見つけるのには役立ちません。それは次の章で学ぶ *vignette* の役割の1つです。

Rは、パッケージ内のオブジェクトのドキュメントを書くための標準的な手段を提供しています。`man/` ディレクトリ内に `Rd` ファイルを書けばよいのです。これらのファイルは、概ね LaTeX をベースとした独自の文法を使っています。そして、HTMLやブレーンテキスト、PDFで表示するために描画されます。これらのファイルを手で書く代わりに、`roxygen2` が特殊な書式で記載したコメントを `.Rd` ファイルに変換してくれます。`roxygen2` が目標としているのは、できるだけ簡単にコードのドキュメントを作成することです。手で `Rd` ファイルを書くよりもたくさんの有利な点があります。

- コードを修正したときに、その部分のドキュメントを修正するのを忘れないように、コードとドキュメントが混在した状態にすることができる。
- ドキュメントを作成しようとしているオブジェクトを `roxygen2` は動的に調査し、従来手で書いていたような定型文を、ある程度省略することができる。
- 異なる種類のオブジェクトのドキュメントの違いを `roxygen2` がまとめてくれるため、細かい点について学ぶ必要が少ない。

2023/6/13
担当:石田(資源解析G)

- Rでパッケージ内のドキュメントは.Rdファイルを手で書くことができる。
- roxygen2 packageを使うと手で.Rdファイルを書くよりも有利な点がある。
 - コードを修正した時にその部分のドキュメントを修正するのを忘れないように、コードとドキュメントが混在した状態にすることができる。
 - .Rdファイルにのみ適用されるmarkup言語を学ぶ必要がなく、markdownを使用することができる。フォーマットだけでなく、自動ハイパーリンク機能により、リンクされたドキュメントの作成が簡単になる。
 - ドキュメントを作成しようとしているオブジェクトをroxygen2は動的に調査し、従来手で書いていたような定型文を、ある程度省略することができる。
 - 異なる種類のオブジェクトのドキュメントの違いをroxygen2がまとめてくれるため、細かい点について学ぶ必要が少ない。

- **documentation workflow**は**roxygen**コメントをソースコードに追加したときに開始される。**roxygen**コメントは`#`ではじめ、通常の`#`コメントと区別する。

```
#' 2つの数字を加算する
#
#' @param x 1つめの数字.
#' @param y 2つめの数字.
#' @returns 数値のvector.
#' @examples
#' add(1, 1)
#' add(10, 1)
add <- function(x, y) {
  x + y
}
```

@: roxygenタグの開始を示すために通常使われる文字。

*RStudio: 通常、最初に関数を書き、次にそのドキュメントを書く。

関数定義ができたなら、その中のどこかにカーソルを置いて、**Code > Insert Roxygen Skeleton**と実行すると、roxygenコメントで始めることができる。

- roxygen commentがある状態でdevtools::document()を実行するとman/add.Rd fileが生成され、次のようになる。

```
% Generated by roxygen2: do not edit by hand
% Please edit documentation in R/add.R

\name{add}
\alias{add}
\title{2つの数字を加算する}
\usage{
add(x, y)
}
\arguments{
\item{x}{1つめの数字}

\item{y}{もう1つの数字}
}
\value{
数値 vector.
}
右に続く
```

```
\description{
2つの数字を加算する
}
\examples{
add(1, 1)
add(10, 1)
}
```

%: 文末まで続く LaTeX コメントの開始を示すために通常使われる文字。

\: LaTeX のエスケープの開始を示すために通常使われる文字

*RStudio: Ctrl/Cmd + Shift + Dを押すあるいはBuild menu or pane からdevtools::document()を実行できる。

- ?add, help(“add”), もしくは example(“add”)を使うと、Rは\alias{add}を含む .Rd fileを探す。次にそのファイルを解析し、HTMLに変換する。下はRStudioでの結果を示している。

add{rvest}

2つの数字を加算する。

Description

2つの数字を加算する。

Usage

add(x,y)

Arguments

x 1つめの数字

y もう1つの数字

Value

xとyの和

Examples

add (1,1)

add (10,1)

*RStudio: 通常、最初に関数を書き、次にそのドキュメントを書く。

関数定義ができたなら、その中のどこかにカーソルを置いて、Code > Insert Roxygen Skeletonと実行すると、roxygenコメントで始めることができる。

- 基本的なroxygen2 workflowの4つのステップ

1.Rにroxygen2 commentsを追加する。

2. roxygen2 comments からRd filesに変換するためにdevtools::document()を実行するか、
Ctrl/Cmd + Shift + D を押す。

3. documentationで?をプレビューする。

4. documentation が望ましい状態になるまでこれを繰り返す。

- roxygen2 comments, blocks, and tagsの説明

roxygen2 comments: #で始まり、関数より前に記述する。

block: 関数の前にあるすべてのroxygenの行。

tags: blockを分割したもの。@tagName tagValueのような形式になっている。tagsの内容はtag名の終わりで始まり、次のtagの開始まで続く。

introduction: 最初のtagの前のテキスト。

tidyverse packagesに入っているlibrary(stringr)での例を以降で挙げる。

- 例.str_unique(){library(stringr)に入っている関数}でのdocumentation

```
# 重複した文字列を除去する
#
# `str_unique()` は重複した値を除去する。
# 重複の測定方法は任意に制御できる。
#
# @param 文字列Input vector. 文字vectorもしくは他のvector
# @param ... 他のoptionsは、重複した文字列間のmatching動作を制御するために使用される。[stringi::stri_opts_collator()]に継承された。
# @returns 通常`string`よりも短い文字vector
# @seealso [unique()], [stringi::stri_unique()]この関数はwraps.
# @examples
# str_unique(c("a", "b", "c", "b", "a"))
#
右に続く
```

```
# # Use ... 追加の引数をstri_unique()に渡す。
# str_unique(c("motley", "mötley", "pinguino", "pingüino"))
# str_unique(c("motley", "mötley", "pinguino", "pingüino"), strength = 1)
# @export
str_unique <- function(string, ...) {
  ...
}
```

ここでintroduction はtitle (“重複した文字列を除去する”)。
introductionの後に5つのtagsが続く: 2つの@params, 1つの@returns, 1つの @seealso, 1つの @examplesと1つの@export。

*RStudio: roxygen commentsの行の長さを手動で管理するのは悩ましいので, Code > Reflow Comment (Ctrl/Cmd+Shift+/)を試用する。

- **introduction**は**title**, **description**と任意に**details**を提供する.

- **title**は最初の文章から引用される.

titleは様々な関数の指標が示され(e.g. `help(package = "somepackage")`), 使用者が複数の関数を閲覧している時に通常は見るだろう。

- **description**は次の文章から引用される.

それは**documentation**の1番上で示され、関数の最も重要な特徴を手短に記述すべきである。

例. `str_detect()` {presence/absenceの一致を検出する関数}での**description**

```
# Detect the presence/absence of a match
#
# `str_detect()` returns a logical vector with `TRUE` for each element of
# `string` that matches `pattern` and `FALSE` otherwise. It's equivalent to
# `grepl(pattern, string)`.
```

- **details**は**description**の後にある.

detailsは任意だが、どんな長さにもできるので、関数の一部の重要な側面を掘り下げたい場合に有用。

introductionにおいては**description**のすぐ後に詳細が来るにもかかわらず、**rendered documentation**(実際に表示される画面のこと?)ではずっと後に登場することに注目。

- `@param` パラメータ名 その作用の説明.

関数の入力、パラメータについて説明する。

説明はパラメータの型(文字列型、数値vector型など)について簡潔に伝えるべきであり、パラメータ名から明らかでない場合は、そのパラメータが何をするものなのかを伝えるべき。

説明は大文字で始まり、ピリオドで終わるべき。

説明は必要に応じて複数行にまたがることできる。

すべてのパラメータについてドキュメントを作成しなければならない。

コンマで区切ることで(スペースは入れない)、複数の引数を1箇所で複数の引数についてドキュメントを作ることができる。

例1. xとyの両方のドキュメントを書きたい場合

```
#' @param x,y Numeric vectors
```

例2. `str_detect()` {presence/absenceの一致を検出する関数}を記述する文字列

```
#' @param 文字列 Input vector. 文字vectorもしくは他のvector
```

- `@inheritParams`.

`tag`を使ってパラメータのドキュメントを再利用する機能。

別の関数から引数を継承することができる。

`@inheritParams`は、関数が実際に使用する引数のうち、すでに文書化されていないものだけを継承するので、一部の引数をlocallyに文書化し、他の引数を継承することができる。

`str_match()` {文字列から一致するgroupsを抽出する関数}はこれを利用して、`str_detect()` の文字列引数の標準documentationを継承しつつ、`pattern` に特化したdocumentationを独自に提供する：

```
#' @inheritParams str_detect
```

```
#' @param pattern 他のstringr関数と異なり、`str_match()`は`vignette(“通常の-expressions”)`として記述された通常の表現のみをサポートする.
```

```
#' そのpatternは取り込まれたgroupを少なくとも1つは含むべきだ.
```

- `@returns` 説明.

関数の出力について説明する。

`library(stringr)`に入っている関数で`@returns documentation`は単刀直入である→ほとんどすべての関数は、入力と同じ長さのある`vector`を返すため。

例1. `str_locate()`{文字列中のパターンの位置を検索する}の出力を記述する

```
#' @returns
```

```
#' * `str_locate()` `string`の各要素での2列1行を持つ整数行列を返す. 最初の列 `start`は開始の位置を  
与え,2列目の`end`は終わりの位置を与える.
```

その上、`@returns`は使用者が重要な`warnings`または`errors`を見るのに良い場所である。

例2. `readr::read_csv()`を記述する文字列は`parsing`で問題がある場合何が起こったかを言及する。

```
#' @returns A [tibble()]. parsing問題がある場合, warningが警告する.
```

```
#'dataset上で[problems()]を呼ぶことによって詳細を読みだすことができる.
```

- `@examples`.

この関数を実際にどのように使うのか、実行可能なRのcodeの情報を提示する。

これはドキュメントの中でも非常に重要→使用者の手助けとなるため。

code例はerror無しで動作しなければならない→R CMD check の中で実行されるまたはpackage websiteがGitHub Actions経由で作られている時などは双方向的ではない状況のため。

`examples`の仕組みは複雑

- 決してerrorがあってはいけないため。
- 4つの異なる状況で実行されるため。
 1. 双方向的に`example()`関数を使っている時。
 2. 自身または別 (例.GitHub Actions内)のコンピュータでR CMD checkを使っている間。
 3. CRANによってR CMD checkが実行されている間。
 4. GitHub Actions または似たものを經由して`pkgdown(websiteを作るpackage) website`が作られる時。

*RStudio: .R scriptsまたは.Rmd / .qmd reportsを準備している時, それを使うためにCtrl/Cmd + EnterまたはconsoleへR codeの行を送る実行buttonが 役立つ.

幸いにも、roxygen comments内で`@examples`の実行と開発の同じworkflowを使うことができる.

- roxygen2は、topicsにまたがるdocumentationの再使用を可能にする多数の機能を提供する.

1. 1つのtopicにおける複数の関数のdocumenting: 初期設定により,各関数は自身のdocumentation topicを得る,しかし、2つの関数がかかなり密接に紐づいている場合は, 1つのtopicに複数の関数のdocumentationを組み合わせてすることができる.

2. 別のtopicから継承しているdocumentation: package内の関数が多いの関連する動作を共有しているかもしれないが、一緒にdocumentにしたいほど密接に紐づけられていない。

3. documentation topicsおよびvignettes間で共有するためのchild documentsの使用: 関数documentationにおいては同じ .Rmd もしくは .md documentが再使用でき, R Markdown child documentsを使用することでvignettesおよびREADME.Rmdが再使用できる.

- `package`のhelp topic.

特殊な番兵(処理範囲の境界に特定の値を目印として置いておくことで、プログラムの構造をシンプルにする手法)“`_PACKAGE`”を記述することによって`package`それ自体のhelp topicを作ることができる.

生じた.Rd fileはtitle, description, 著者の一覧および有用なURLsを含むDESCRIPTIONから構文解析された情報が自動的に取り込まれる.

このhelp topicは全ての他のtopicsとともに出現する、その上これは `package?pkgname`と一緒にアクセス可能.

dummy file `R/{pkgname}-package.R`の中でpackage-level documentationをset upするために`usethis::use_package_doc()`を呼び出すことを推奨する, このcontentsはこのようになるだろう:

```
#' @keywords internal
"_PACKAGE"
```

`R/{pkgname}-package.R` fileがどのように見えるかの例:

```
#' @keywords internal
"_PACKAGE"
# 下記のblockは自動的に管理するためにusethisによって使われる
# roxygen namespace tags. 慎重に修正せよ!
## usethis namespace: start
#' @importFrom glue glue_collapse
## usethis namespace: end
NULL
```

• テキスト書式参照表 roxygenタグでは.Rd形式のテキスト書式を使う。

文字の書式

<code>\emph{italics}</code>	イタリック
<code>\strong{bold}</code>	太字
<code>\code{r_function_call(with="arguments")}</code>	引数"arguments"を指定したRの関数{r_function_call}呼び出し

他のドキュメントへのリンク

<code>\code{\link{function}}</code>	同じパッケージ内への関数のリンク
<code>\code{\link[MASS]{abbey}}</code>	他のパッケージ関数へのリンク
<code>\link[=dest]{name}</code>	destにリンクするが、表示はnameにする

webへのリンク

<code>\url{http://rstudio.com}</code>	特定のURL
<code>\href{http://rstudio.som}{Rstudio}</code>	リンクテキストを指定したURL
<code>\email{Hadley@@rstudio.com}(@が2つあるのに注意)</code>	メールアドレス

• テキスト書式参照表 続き。

リスト

順序付き(番号付き)リスト	<pre>#'\enumerate{ #\item 1つめの項目 #\item 2つめの項目 # }</pre>
順序なし(箇条書き)リスト	<pre>#'\itemize{ #\item 1つめの項目 #\item 2つめの項目 # }</pre>
定義(名前付き)リスト	<pre>#'\describe{ #\item {One}1つめの項目 #\item {Two}2つめの項目 # }</pre>

数式

<code>\eqn{a+b}</code>	インライン式
<code>\deqn{a+b}</code>	ブロック表示の式

- 表 表は`\tabular{}`で作ることができ、引数が2つある。
 - 列の並びを1文字で指定して調整する(l=左寄せ、r=右寄せ、c=中央寄せ)。
 - テーブルの内容の整形。列を`\tab`で区切り、`\cr`で改行する。

Rのデータフレームを適切な書式に変換する例：

```
tabular <- function(df,...){
  stopifnot(is.data.frame(df))

  align <- function(x) if (is.numeric(x)) "r" else "l"
  col_align <- vapply(df,align,character(1))

  cols <- lapply(df,format,...)
  contents <- do.call("paste",
    paste("//tabular{",paste(col_align,collapse=""),"} {\n",
    contents, "\n} \n",sep = "")
}
```

右に続く

```
cat(tabular(mtcars[1:5,1:5]))
#>  \tabular{rrrrr}{
#>  21.0\tab6\tab160\tab110\tab3.90\cr
#>  21.0\tab6\tab160\tab110\tab3.90\cr
#>  22.8\tab4\tab108\tab93\tab3.85\cr
#>  21.4\tab6\tab258\tab110\tab3.08\cr
#>  18.7\tab8\tab360\tab175\tab3.15
#> }
```